Sri Lanka Institute of Information Technology



Green DevOps - Energy efficient CI/CD Pipieline

R26-SE-027

Green DevOps - Energy efficient CI/CD Pipieline

The Real-World Problem

- Rapid growth of cloud-native systems and CI/CD pipelines creates green debt.
- DevOps pipelines prioritize speed over energy efficiency.
- Small code changes trigger full builds and tests.
- This causes resource waste, high energy usage, and increased carbon emissions. [1][2]

Credible Evidence

- Carbon Footprint: ICT sector produces 2–4% of global CO_2 emissions.[1]
- Resource Waste: About 90% CPU time wasted during full rebuilds.[2]
- Economic Impact: 35% annual increase in cloud costs.[1]

Why our solution is valuable

Aspect	Existing Systems / Tools
Carbon Monitoring	**Cloud Carbon Footprint, Green Software Foundation SCI, AWS Customer Carbon Footprint To**
Carbon-Aware Scheduling	**Carbon Aware SDK (Microsoft), ElectricityMaps-based schedul**
Resource Scaling	**Kubernetes Horizontal Pod Autoscaler (HPA)**

Group Members And their Respective Components

1. Release Management (Mendis J.D.L. – IT22110084)

2. Deployment Stage (Rathnayake D.M.B.H. – IT22149930)

3. Operations Stage (Thassara M.P.M. – IT22281432)

4. Monitoring Stage (Kodithuwakku I.P. – IT22189776)

Release stage Component
IT22110084 | Mendis J.D.L.

Contribution

- **Optimize CI/CD releases using change impact analysis**
- **Reduce redundant builds through artifact reuse and incremental builds**
- **Enable risk-based selective test execution**
- **Improve pipeline efficiency and reduce resource usage**

Novelty

- **Intelligent release decision layer in CI/CD pipelines**
- **Integration of change analysis + artifact reuse**
- **Sustainability-driven release optimization**

Release stage Component

Problem in Traditional CI/CD

Traditional pipelines typically perform

- Full system builds for every commit
- Execution of the entire test suite
- Deployment regardless of change scope

Leads to

- High computational overhead
- Increased energy consumption
- Higher carbon emissions from cloud infrastructure

Main Objective

To develop a sustainable release management component that reduces unnecessary computational workload in CI/CD pipelines and enables carbon-aware scheduling of build tasks to minimize energy consumption and environmental impact.

Sub Objectives

1. Reduce Redundant Build Processes
2. Optimize Test Execution
3. Integrate Carbon-Aware Scheduling

Release stage Component

Research Methodology

- DevOps pipeline optimization
- Change impact analysis and incremental builds
- Artifact reuse to reduce redundant computation
- Carbon aware scheduling using XGBoost model.

Technologies Used

- CI/CD Platforms (Jenkins / GitHub Actions)
- Git commit analysis
- Maven / Gradle incremental builds
- Nexus artifact repository
- Docker containerized build environments

```
DEPENDENCY_GRAPH = {
   "common-lib": ["auth-service", "order-service", "payment-service"],
   "auth-service": [],
  "order-service": ["checkout-service"],
  "payment-service": ["checkout-service"],
   "checkout-service": []
}
```

Release stage Component

Validation & Evaluation

- Compare optimized pipeline with traditional CI/CD
- Metrics: pipeline execution time, rebuilt modules, CPU usage

Data, Security & Ethics

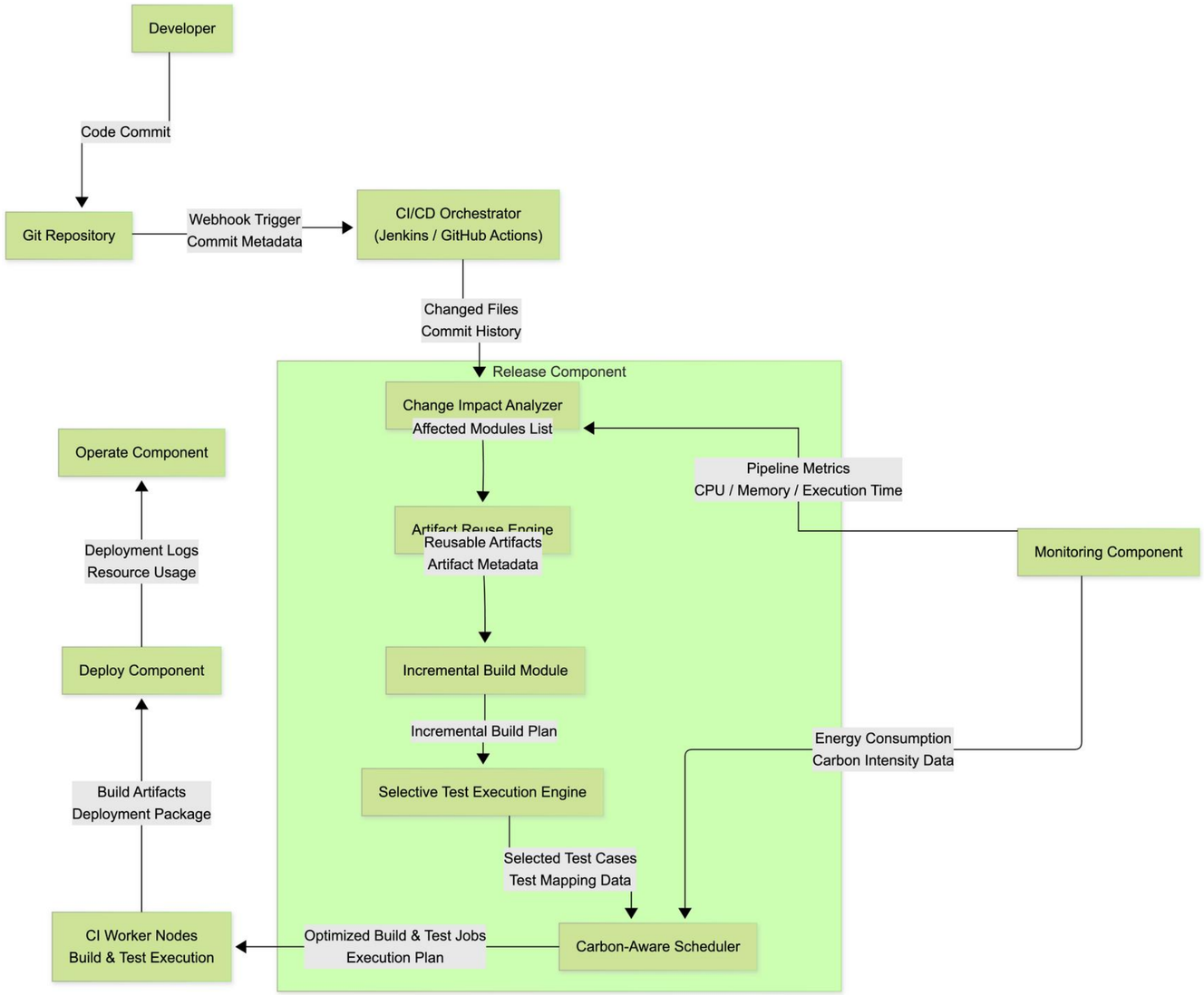
- Uses CI/CD pipeline execution data
- No personal or sensitive data processed
- Secure repository and artifact access

Release stage Component

CI/CD Infrastructure
Git Repository (GitHub / GitLab) – stores source code and triggers CI/CD pipelines
CI/CD Orchestrator (Jenkins / GitHub Actions) – manages pipeline execution
CI Worker Nodes – execute build and test jobs

Build and Artifact Management
Build Tools (Maven / Gradle / npm) – compile and package the application
Artifact Repository (Nexus / Artifactory) – stores generated build artifacts

Sustainability Dependency
Carbon Intensity API (e.g., WattTime) – provides carbon intensity data used by the scheduler



Deploy Stage

IT22149930 Component Rathnayake D.M.B.H.

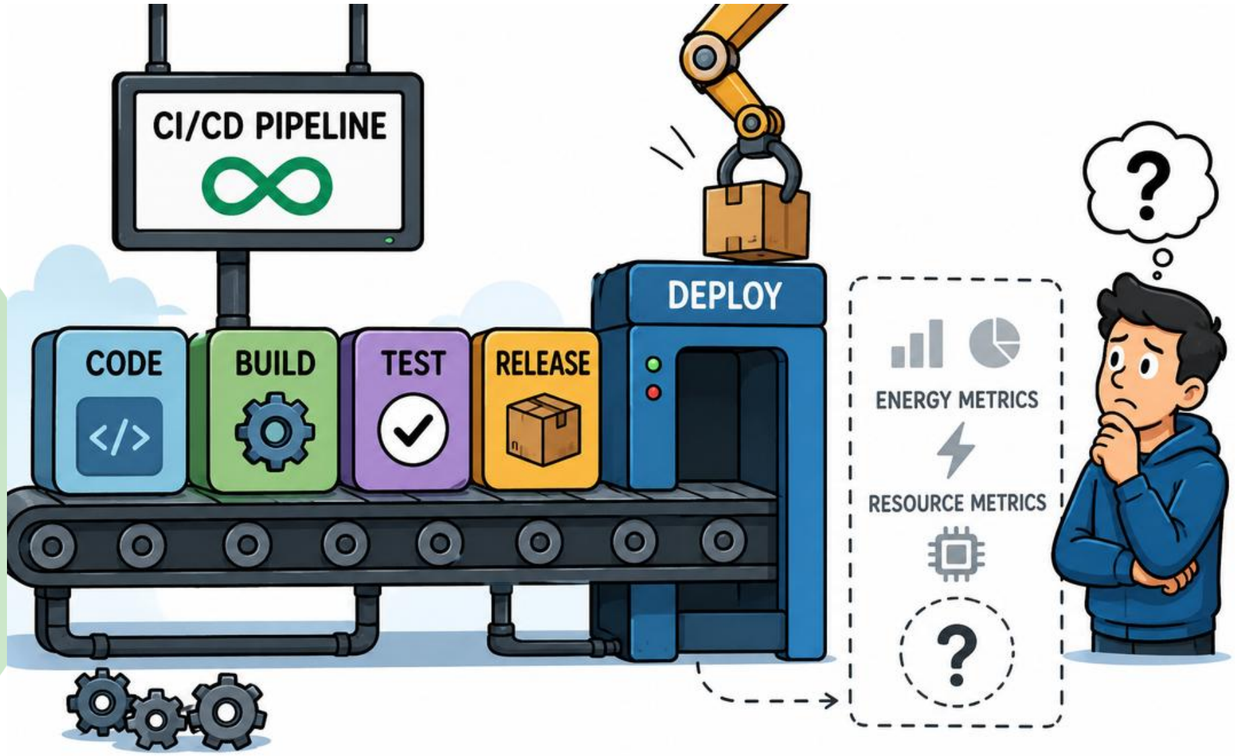
Contribution

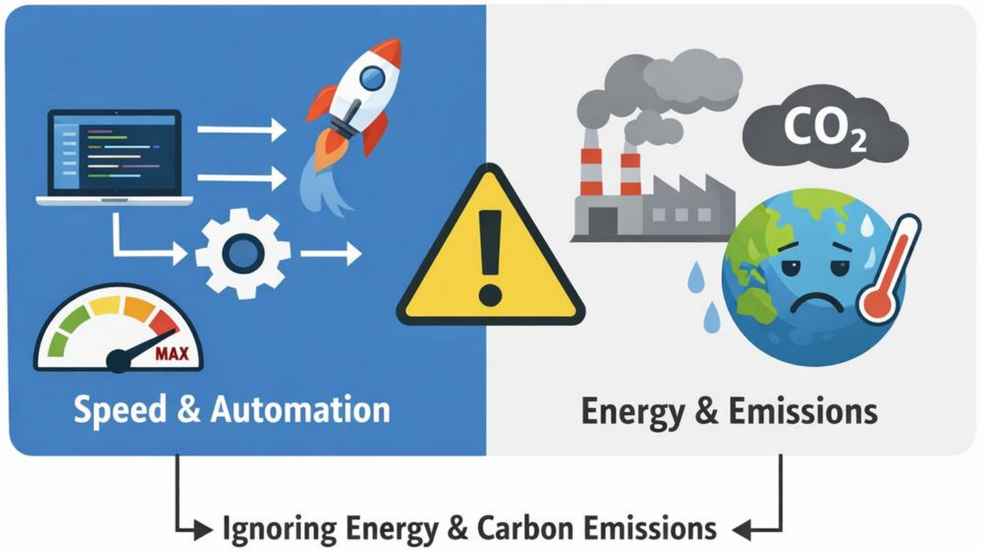
introducing an energy awareness and **AI-driven decision-making** into the CI/CD pipeline.

Novelty of the Solution

- Deployment Energy Profiler - Measure deployment energy usage
- AI-based Green Deployment Decision Engine - Intelligent deployment planning
- Energy-optimized strategy selection

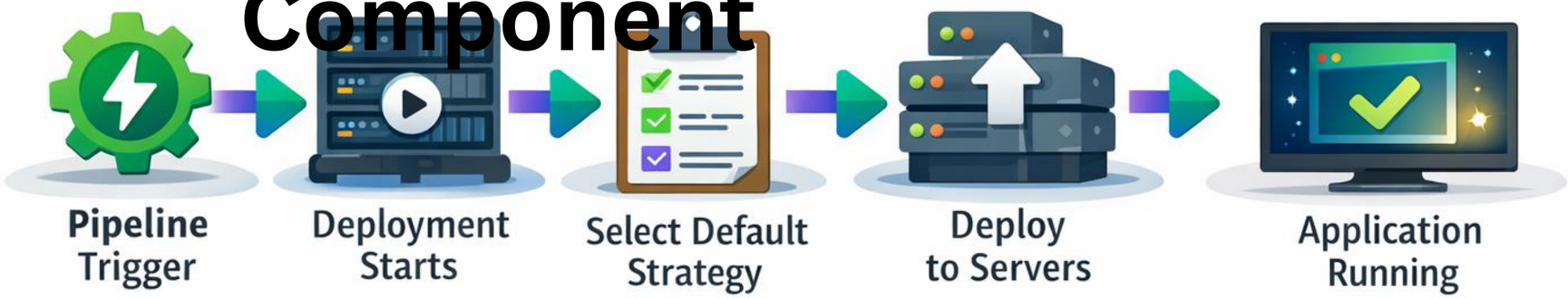
Deploy Stage Component



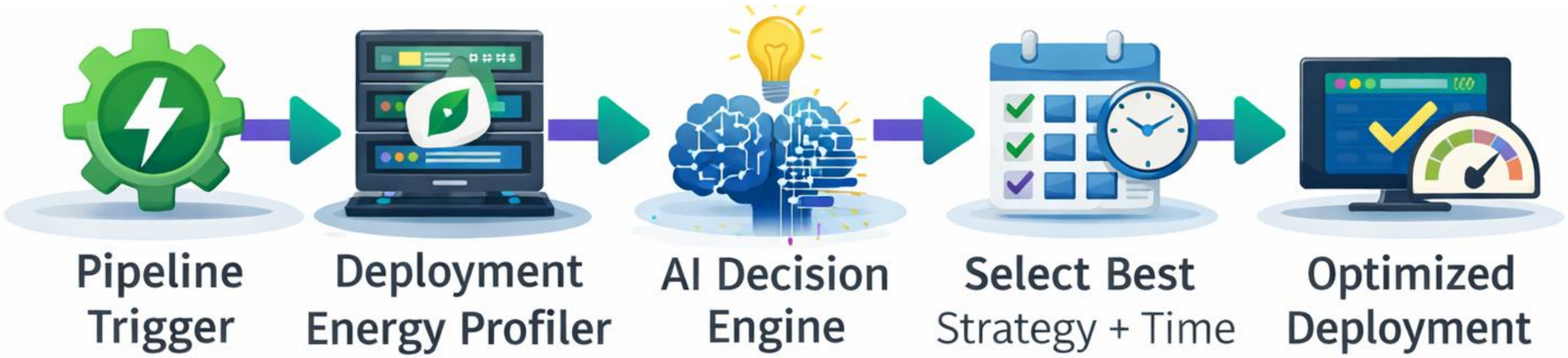


Deploy Stage Component

Traditional Pipeline



Proposed Feature

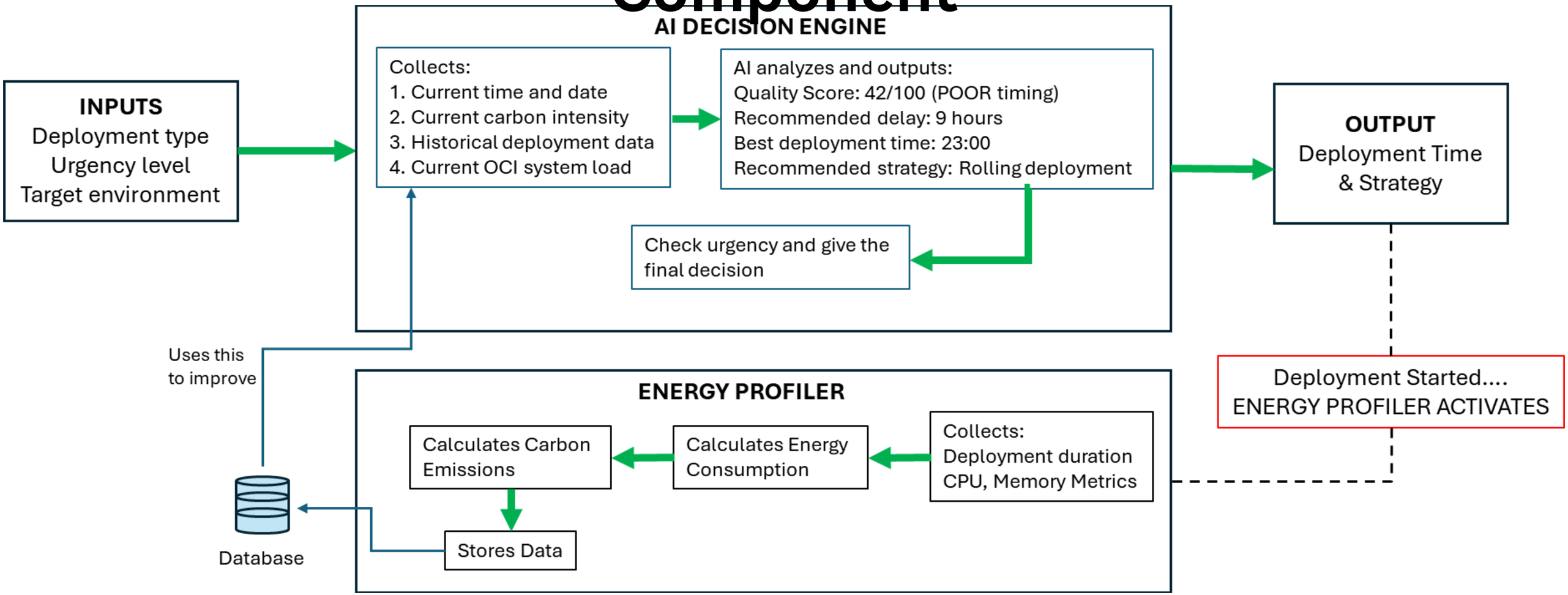


Research Methodology (Knowledge Domains)

- Green Computing
- DevOps & CI/CD
- Cloud Computing
- Machine Learning

[3][1]

Deploy Stage Component



Deploy Stage Component

Tools & Technologies











- Supervised Learning Algorithm

Validation & Evaluation

- Compare energy consumption
- Measure carbon emission reduction
- Evaluate ML model prediction accuracy
- Test with different deployment strategies

Data, Security & Ethics

- Secure storage in cloud database
- No personal or sensitive user data
- Azure public dataset
- Deployment logs (during experiment)

Operation Stage Component

IT22281432 | M.P.M Thassara

Individual Contribution

- Designed a sustainable DevOps operation module.
- Integrated AI-driven workload prediction.
- Implemented carbon-optimized scaling strategy.
- Added energy-aware job prioritization.



Novelty

- Adds carbon-aware decision making to DevOps operations
- Integrates AI workload prediction with energy-efficient scaling and job prioritization.

Operation Stage Component

Problem Definition

- Scaling based only on CPU and memory.
- Energy and carbon impact not considered.
- Causes resource waste and higher environmental impact



Research Objective

•Develop a Green DevOps operation system that optimizes resource usage while reducing carbon emissions.

Sub Objectives

•Predict future workload using AI
•Optimize scaling decisions based on energy usage
•Prioritize jobs to reduce unnecessary resource consumption

Operation Stage Component

Domain Knowledge

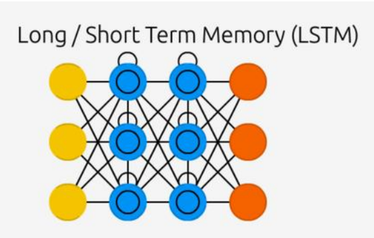
- Uses Green DevOps to reduce cloud carbon emissions
- Uses Kubernetes for intelligent resource management
- Applies AI/ML for workload prediction
- Optimizes energy through sustainable computing

Tool And Tecnology

ORACLE Cloud Infrastructure

kubernetes Metrics Server

kubernetes



python™

Methodology

Step	Description
Data Collection	Collect Kubernetes metrics (CPU, memory, request rate)
Data Preprocessing	Clean and normalize monitoring data
AI Workload Prediction	Forecast workload using LSTM model
Carbon Optimization	Estimate energy usage and carbon emissions
Job Prioritization	Delay low-priority background tasks
Scaling Decision	Choose the most energy-efficient scaling strategy

Operation Stage Component

Validation and Evaluation

- **Prediction Accuracy** – Predicted vs actual workload
- **Carbon Reduction** – Energy usage before vs after
- **Resource Efficiency** – CPU & memory utilization
- **Response Time** – Performance impact of scaling
- **Autoscaling Comparison** – vs Kubernetes HPA

Data Sources

- Kubernetes monitoring data
- Prometheus metrics database
- Simulated workload datasets for testing

Security

- Secure API communication
- Controlled data access
- Protected cloud environment

Ethical Considerations

- No personal data used
- Analyzes only system metrics
- Ensures sustainable cloud computing

Operation Stage Component

System Workflow

1.Collect monitoring data (CPU, memory, request load)

2.Predict future workload using AI

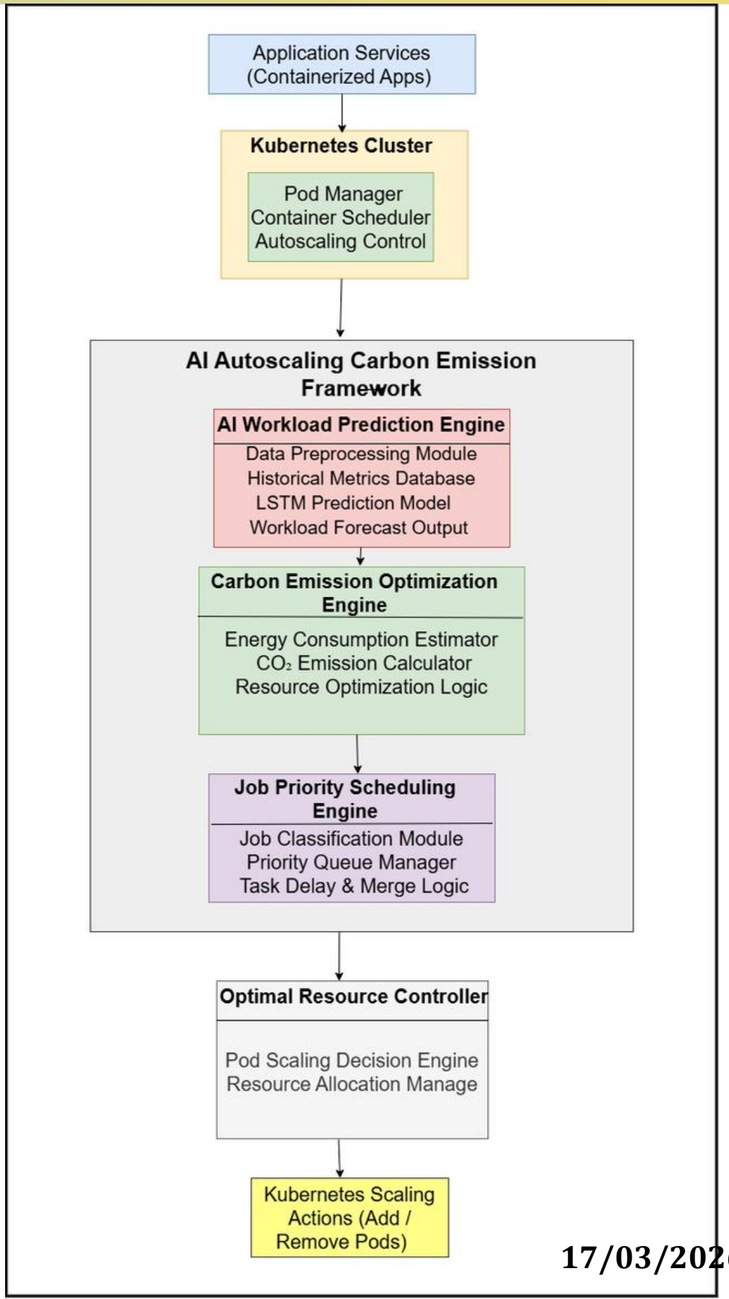
3.Analyze energy usage and carbon impact

4.Apply job priority scheduling

5.Make optimal pod scaling decision in Kubernetes

Dependencies

1. Kubernetes cluster
2. Monitoring tools (metrics server)
3. AI prediction model
4. Cloud Infrastructure (Oracle Cloud)



Monitor Stage Component

IT22189776 | I.P. Kodithuwakku

Contribution

- Collect CI/CD pipeline execution metrics
- Estimate energy consumption
- Calculate carbon emissions
- Generate sustainability datasets

Novelty

- Pipeline-level sustainability monitoring
- Energy and carbon estimation integrated into DevOps workflow

Monitor Stage Component

Problem

- CI/CD pipelines run thousands of automated builds daily
- Current DevOps monitoring tools focus on performance and reliability
- Environmental impact of CI/CD pipelines is not measured

Research Gap

- Lack of pipeline-level energy and carbon monitoring

Novelty:

- Sustainability-aware monitoring framework
- Converts pipeline metrics → energy → carbon emissions

Main objective:

- Develop a sustainability monitoring framework for CI/CD pipelines

Sub-objectives:

1. Collect pipeline execution metrics
2. Estimate energy consumption
3. Calculate carbon emissions
4. Generate sustainability datasets

Monitor Stage Component

Domain

Software Engineering – DevOps & Sustainable Computing

Research Method

Design Science Research (DSR)

Steps:

1. Problem Identification
2. Artifact Design (Monitoring Framework)
3. Implementation
4. Evaluation



Technologies Used

- Python (monitoring scripts)
- psutil library (resource monitoring)
- CI/CD platforms (GitHub Actions / Jenkins)
- ElectricityMaps API (carbon intensity)
- Cloud Infrastructure (OCI)

Why These Technologies?

- Compatible with modern DevOps pipelines
- Lightweight monitoring with minimal overhead
- Real-time carbon estimation

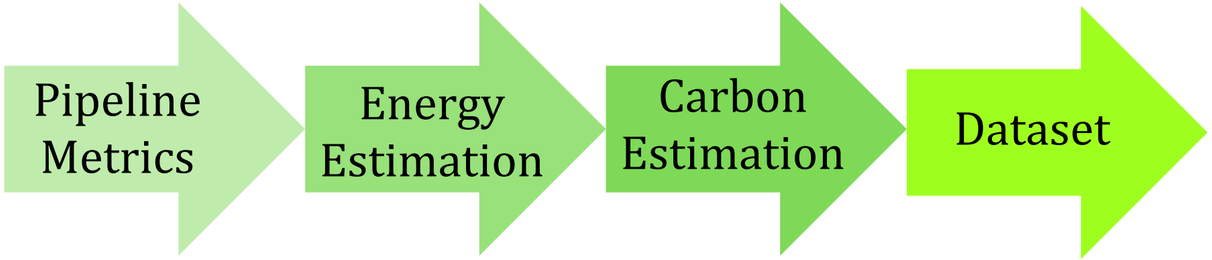
Monitor Stage Component

Validation Strategy

- Run CI/CD pipeline experiments
- Monitor resource usage
- Estimate energy consumption
- Compare pipeline stage energy patterns

Data Availability

- CI/CD execution logs
- CPU & memory metrics
- Carbon intensity datasets



Evaluation Metrics

- Energy consumption per pipeline stage
- Estimated carbon emissions
- Resource utilization trends
- Monitoring system performance overhead

Security & Ethics

- No personal or sensitive data collected
- Only system performance metrics
- Uses publicly available datasets

Monitor Stage Component

Monitoring component modules:

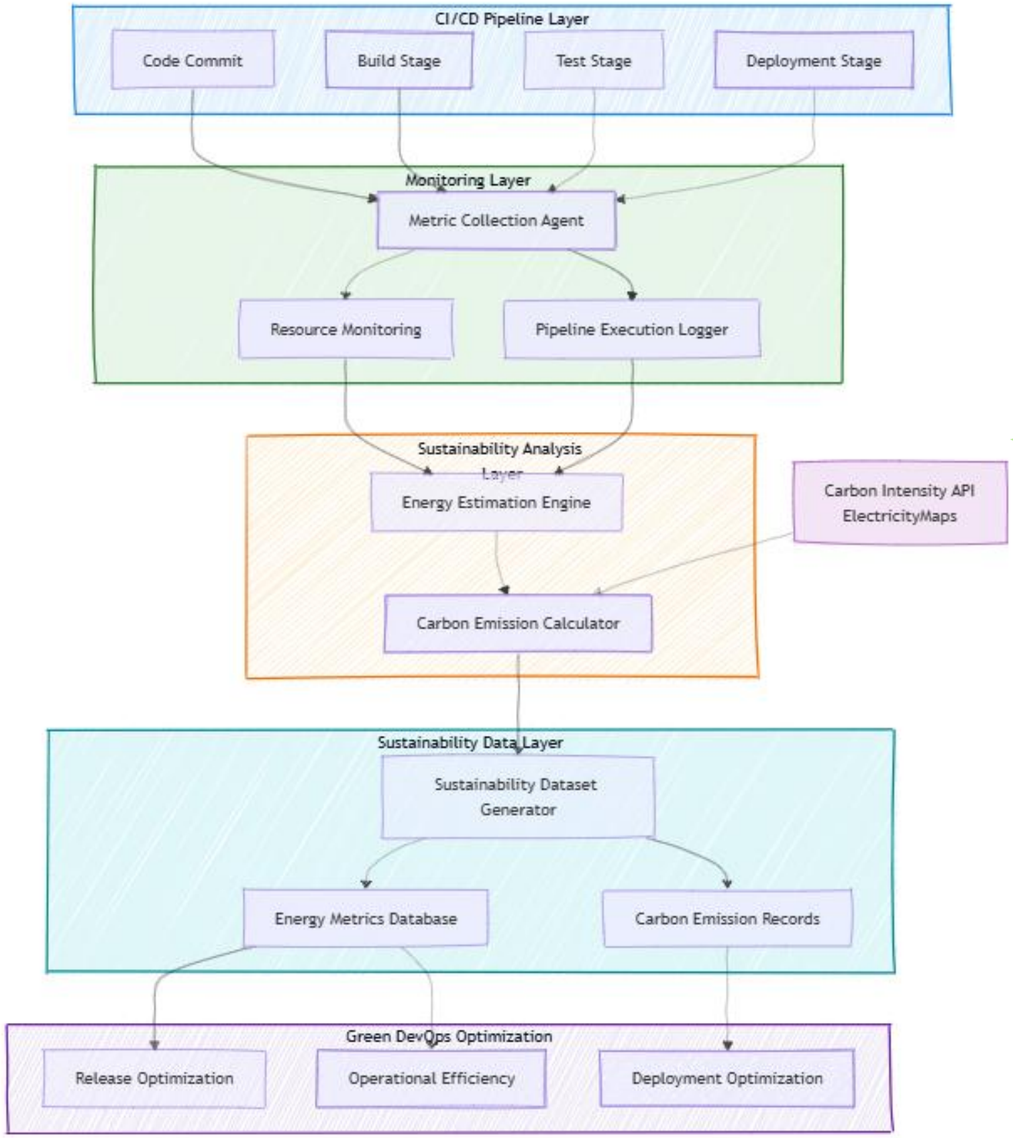
1. Metric Collection Module
2. Energy Estimation Module
3. Carbon Estimation Module
4. Sustainability Dataset Generator

Output

- Carbon emission estimates
- Pipeline energy insights
- Sustainability dataset

Dependencies

- CI/CD pipeline environment
- Cloud infrastructure
- Carbon intensity API



Green DevOps - Energy efficient CI/CD Pipieline

- Ability of Commercialization
  - Closes the measurement gap by tracking Software Carbon Intensity (SCI) in DevOps pipelines.
  - Uses AI-based workload prediction and carbon-aware scheduling, unlike traditional tools such as

Category	Description
Development Cost	AI model development, system architecture, software impleme
Infrastructure & Hardware	Cloud clusters, prototype hardv (Raspberry Pi, sensors)
Maintenance & Marketing	System maintenance, promotio deployment support

Target Market

- Cloud-native companies
- Large enterprises using CI/CD pipelines

Customer S

- DevOps / SRE Engineers
- Sustainability Officers

Green DevOps - Energy efficient CI/CD Pipieline

	Semester 1							Semester 2					
Task	Dec 2025	Jan 2026	Feb 2026	Mar 2026	Apr 2026	May 2026	Jun 2026	Jul 2026	Aug 2026	Sep 2026	Oct 2026	Nov 2026	Dec 2026
Topic Selection & Idea Discussion	■												
Refer Existing Research Papers		■											
Literature Review		■	■										
Research Gap Identification			■										
System Architecture Design				■									
Research Proposal Writing				■	■								
Proposal Submission & Presentation					■								
Dataset Collecting						■							
Start Development					■	■	■	■					
Progress Presentation 1							■		(Ctrl)				
Testing Individual Components								■					
Bug Fixing								■	■				
Integrate Each Component									■				
Progress Presentation 2										■			
Whole System Testing & Evaluation										■	■		
Final Report & Website											■	■	
Final Presentation & Viva												■	
Research Paper Submission												■	
Publication Evidence Submission													■

Green DevOps - Energy efficient CI/CD Pipieline

WBS ID	Task	Description
1	Research Planning	Initial planning and understanding of Green DevOps co
1.1	Topic Selection	Identify research topic related to Green DevOps and er
1.2	Literature Review	Review existing research on DevOps energy optimizatio
1.3	Research Gap Identification	Identify limitations in existing systems and define resea
2	System Design	Design the architecture for the proposed Green DevOp
2.1	Architecture Design	Design high-level system architecture and component i
2.2	Proposal Writing	Prepare research proposal document
2.3	Proposal Presentation	Submit proposal and present to evaluation panel
3	Data Preparation	Collect and prepare operational data for AI models
3.1	Dataset Collection	Collect system metrics such as CPU, memory, request r
4	AI Model Development	Develop workload prediction model using AI
4.1	Prediction Engine Development	Implement AI workload prediction engine using LSTM
4.2	Model Training	Train and improve the AI model using historical workloa
5	Optimization Module Development	Develop modules to reduce carbon emissions in CI/CD
5.1	Carbon Optimization Engine	Estimate energy usage and carbon emission for scaling
5.2	Job Priority Scheduling	Classify CI/CD jobs based on priority and optimize sche
6	System Implementation	Integrate developed modules with cloud infrastructure
6.1	AI Model Testing	Test prediction accuracy and performance
6.2	System Integration	Integrate prediction engine, carbon optimizer and sche
7	Evaluation	Evaluate system performance and optimization results
7.1	System Testing	Perform functional testing and performance evaluation
8	Final Deliverables	Final documentation and presentation
8.1	Final Report Writing	Prepare final thesis and documentation
8.2	Final Presentation	Present research results and system demonstration
8.3	Research Paper Submission	Submit research paper to conference/journal
8.4	Publication Evidence Submission	Submit proof of publication

COMMERCIALIZATION & SUSTAINABILITY





Thank You